

HelixMemory

HelixMemory

چهار موتور پیشرو، درهم آمیخته — AI یک حافظه مغزی برای عوامل

خلاصه

HelixMemory (Mem0، Cognee، Letta، Graphiti) است که چهار سیستم حافظه پیشرو Go SDK یک را در یک موتور حافظه شناختی واحد ادغام می کند که به طور همزمان در آن ها (Graphiti، Letta، Cognee، Mem0) یک لایه حافظه پایدار، بدون تکرار AI جستجو کرده و نتایج را تلفیق می نماید. این سامانه به برنامه های و بلز مرتب شده ارائه می دهد، به جای چهار لایه ناهمگونگ

توضیح کوتاه

HelixMemory را در Graphiti و Mem0، Cognee، Letta است که Go SDK یک را در یک موتور حافظه شناختی واحد ادغام می کند. این سامانه داده ها را به طور AI یک موتور حافظه شناختی واحد برای برنامه های هوشمندانه مسیریابی می کند، در تمام پایگاه های پشتیبان به طور موازی جستجو می نماید و نتایج را از طریق یک خط لوله سه مرحله ای جمع آوری، حذف تکرار و بلز مرتب سازی تلفیق می کند.

توضیح مفصل

Go است که به صورت یک AI یک موتور حافظه شناختی واحد برای برنامه های HelixMemory (به بالا) ارائه می شود. فرض ۱٫۲۵ Go نسخه، digital.vasic.helixmemory (ماژول) Go SDK بنیادین آن این است که هیچ پروژه حافظه واحدی هرگز در همه زمینه ها بهترین نخواهد بود — بنابراین به جای پیاده سازی مجدد حافظه از صفر و به ارث بردن نقاط کور یک پروژه، چهار سیستم پیشرو را برای استخراج پویای Mem0: هماهنگ کرده و به هر یک اجازه می دهد در زمینه تخصصی خود برخشد برای گراف های دانش معنایی ساخته شده از طریق خطوط لوله Cognee، حقایق و مدیریت ترجیحات برای محیط اجرایی عامل دارای حالت با بلوک های حافظه قابل ویرایش و محاسبات ECL، Letta، Graphiti زمان خواب، و برای گراف دانش دوبعدی زمانی که در بلز چگونگی تغییر حقایق در گذر زمان Graphiti زمان خواب، و استدلال می کند.

موتور تلفیق است که این چهار مخزن مستقل را به یک مغز واحد تبدیل می کند. در مسیر نوشتن، هر حافظه ورودی بر اساس محتوا طبقه بندی شده و به پایگاه پشتیبان مناسب ترین برای نگهداری آن هدایت می شود. در مسیر خواندن، یک پرسوجو به طور همزمان در تمام پایگاه های پشتیبان پخش شده و نتایج خام وارد یک خط لوله تلفیق سه مرحله ای می شوند — جمع آوری، حذف تکرار، سپس بلز مرتب سازی بین منابع — به گونه ای که فراخواننده هرگز با چهار مجموعه نتیجه پراکنده و همپوشان مواجه نمی شود، بلکه

تنها یک پاسخ مرتب شده و تمیز دریافت می کند. قطع کننده های مدار هر پایگاه پشتیبان را احاطه کرده اند تا در صورت بروز مشکل، به طور منظم از کار افتاده و سایر پایگاه ها به ارائه خدمات ادامه دهند، بدون آنکه قابل MemoryStore کل لایه حافظه را با خود به پایین بکشند. از آنجا که این موتور یک رابط جایگزینی را پیاده سازی می کند، به عنوان جایگزین مستقیم یک ارائه دهنده حافظه ساده عمل کرده و جزئیات مسیریابی و تلفیق را Prometheus نیازی به بازطراحی فراخواننده نیست — و معیارهای برای مشاهده پذیری کامل آشکار می سازند.

از Helix مجموعه گسترده تر، HelixAgent به عنوان لایه حافظه برای HelixMemory ساخته شده و نظم آزمون ضدفریب این خانواده را به حوزه حافظه گسترش می دهد: یک AI، برنامه های — اجراگر چالش درون فرایندی مسیرهای کد واقعی تولید را — مسیریابی، تلفیق، مترجم، قطع کننده مدار به کار می گیرد، در حالی که یک بسته بندی جهش جفت شده به طور عمدی ثابت ها را تغییر می دهد تا ثابت کند آزمون ها واقعاً در صورت نقص منطق شکست می خورند، بنابراین یک مجموعه سبز معنای واقعی دارد.

چرا آن را ساختیم

محتوا

به حافظه ای بادوام و باکیفیت نیاز دارند، اما اکوسیستم حافظه ها تکه تکه است — هر AI عامل های در یک زمینه قوی و در زمینه های دیگر (Mem0، Cognee، Letta، Graphiti) پروژه حافظه یک سطح HelixAgent برای این ساخته شد که به HelixMemory ضعیف عمل می کند. حافظه واحد بدهد که نقاط قوت همه آنها را بدون اجبار به وابستگی به هیچ کدام ترکیب کند.

چرا انقلابی است

این راهکار انتخاب اجباری را پایان می دهد. چهار سیستم حافظه که معمولاً برای تصاحب یک جایگاه با هم رقابت می کنند، پشت یک رابط واحد به مکمل هایی برای یکدیگر تبدیل می شوند — به این ترتیب یک برنامه می تواند همزمان از استخراج پویای حقایق، گراف های دانش معنایی، حافظه عامل های دلزای حالت و استدلال دوزمانی بهره مند شود، در حالی که حذف تکراری ها و رتبه بندی مجدد منابع به طور خودکار انجام می شود. آنچه پیش از این عملی نبود، این است که پرسش «کدام موتور حافظه را باید به شما امکان می دهد همزمان از همه HelixMemory: انتخاب کنیم؟» را یک دروازه کاذب بدانیم یکپارچه و بدون به ارث بردن نقاط کور هیچ MemoryStore نقاط قوت آنها بهره مند شوید، پشت یک موتور یا تعهد به وابستگی به آن.

نوآوری ها

- که به جای محدود (جمع آوری → حذف تکراری ها → رتبه بندی مجدد منابع) ادغام چندپشتیبان کردن فراخواننده به یک حافظه واحد، یک مجموعه نتیجه رتبه بندی شده ارائه می دهد.
- که هر حافظه را بر اساس محتوا طبقه بندی کرده و به موتوری می فرستد مسیره های هوشمند نوشتن. که بهترین گزینه برای نگهداری آن است، تا داده های مناسب در حافظه مناسب قرار گیرند.
- از طریق قطع کننده های مدار برای هر پشتیبان — خرابی یک موتور جداسازی تخریب تدریجی می شود و کشنده نیست، و بقیه به خدمت رسانی ادامه می دهند.
- که حافظه را در دوره های بیکاری بازسازی (Letta از طریق) تلفیق محاسبات در زمان استراحت می کند، نه فقط در زمان پرس و جو.

- یک اجراگر چالش روی کد واقعی تولیدی، همراه با یک بسته‌کننده جهش که باید: تأیید ضد-فریب، در صورت نقض یک ثابت شکست بخورد — اثبات اینکه دروازه تست یک بررسی واقعی است، نه یک توتولوژی.

بزرگ‌ترین چالش‌های فنی و راه‌حل‌های ما

- هر موتور حافظه را به شکل — تلفیق چهار پشتیبان ناهمگن در یک مجموعه نتیجه منسجم خاص خود بلزمی‌گرداند، و ادغام ساده آنها منجر به تکراری‌ها و رتبه‌بندی‌های ناسازگار می‌شود. این مشکل با یک موتور تلفیق تایپ‌شده حل شد که داده‌ها را از منابع مختلف جمع‌آوری می‌کند، همپوشانی‌ها را حذف می‌کند و همه را بر اساس یک معیار مشترک رتبه‌بندی می‌کند، با ثابت تلفیق‌شده که در تست‌ها تأیید می‌شود تا ادغام نتواند به‌طور پنهانی نتایج را از دست بدهد یا دو بار شمارش کند.
- عدم دسترسی به یک موتور حافظه نباید کل لایه را — پایداری در برابر خرابی یک پشتیبان متوقف کند. این مشکل با قطع‌کننده‌های مدار برای هر پشتیبان حل شد که بر اساس یک ماشین حالت بسته → باز (پس از آستانه خرابی) → نیمه‌باز (پس از زمان انتظار) عمل می‌کنند، موتور معیوب را جداسازی کرده و تازمان بازیابی آن، از پشتیبان‌های سالم به خدمت‌رسانی ادامه می‌دهند.
- یک مجموعه تست سبز — اثبات کارکرد واقعی منطق حافظه، نه فقط کامپایل شدن آن بی‌معنی است اگر تست‌ها نتوانند شکست بخورند. این مشکل با یک اجراگر چالش درون‌فرآیندی حل شد که کد واقعی تولیدی (مسیردهی، تلفیق، مترجم، قطع‌کننده مدار) را اجرا می‌کند و همراه با یک بسته‌کننده جهش که ثابت‌ها را تغییر می‌دهد و انتظار شکست تست‌ها را دارد، تا ثابت شود دروازه یک بررسی واقعی است، نه یک توتولوژی.

محتوا

پشته فناوری

- **Go** (نسخه ۱.۲۵ به بالا) — انتخاب شده زیرا توزیع خوانش SDK هسته اصلی — آن را کم‌هزینه می‌کنند، در Go موازی در چهار بک‌اند یک مسئله همروندی است و گوروتین‌های برای کل سیستم فراهم می‌آورد که (MemoryStore) حالی که انواع اینترفیس آن یک درز تمیز فراخوانندگان می‌توانند به آن تکیه کنند.
- **Mem0** — بک‌اند استخراج پویای حقایق و مدیریت ترجیحات؛ برای بخش «کاربر واقعاً چه چیزی را ترجیح می‌دهد / چه حقایقی آشکار شده‌اند» از حافظه استفاده می‌شود.
- **Cognee** — ساخته شده؛ برای ECL بک‌اند گراف دانش معنایی که بر پایه پایپ‌لاین‌های نگهداری دانش ساختاریافته و مرتبط به جای حقایق مسطح استفاده می‌شود.
- **Letta** — بک‌اند زمان‌اجرای عامل‌های دارای حالت با بلوک‌های حافظه قابل ویرایش و محاسبات زمان خواب؛ در مواردی استفاده می‌شود که حافظه باید به‌عنوان حالت زنده عامل باقی بماند و در دوره‌های بیکاری تثبیت شود.
- **Graphiti** — بک‌اند گراف دانش دوزمانی؛ برای استدلال درباره چگونگی تغییر حقایق و روابط. در طول زمان، نه فقط مقدار فعلی آن‌ها، استفاده می‌شود.
- **PostgreSQL + Neo4j + Redis** — انبارهای داده واقعی که بک‌اندها بر روی make infra-start آن‌ها اجرا می‌شوند و برای تست یکپارچگی واقعی با دستور برپا شده‌اند تا make infra-start آن‌ها اجرا می‌شوند و برای تست یکپارچگی واقعی با دستور مجموعه تست‌ها زیرساخت زنده را به جای ماکت‌ها آزمایش کند.
- **Prometheus** — معیارها و قابلیت مشاهده‌پذیری که از طریق پایپ‌لاین تلفیق متصل شده‌اند تا مسیریابی و رفتار تلفیق در محیط عملیاتی قابل اندازه‌گیری باشد، نه یک جعبه سیاه.

- که در جای خود نگه (`helixmemory_`) سطح رشته‌های نام‌گذاری شده — درز مترجم بین‌المللی داشته شده تا هر لایه کاربرمحور آینده بتواند بدون تغییرات پسینی در هسته بومی‌سازی شود.

وضعیت و یادداشت‌های شفافیت

- ساخته شده HelixAgent در حال کار است؛ به‌عنوان لایه حافظه برای SDK. وضعیت: بتا است.
- شناسایی نشد — تأیید نشده / اعلام GitHub API هیچ مجوزی از طریق مجوز: نامشخص نشده است.
- نگاشت می‌شود. ارقام دقت ذکر شده در memory به مخزن «HelixMemory» نام نمایشی ادعاهای فروشندگان بالادستی هستند، نه اندازه‌گیری‌های README فایل HelixMemory، و در اینجا حذف شده‌اند.

اصلی-Helix: اولویت